



UNIVERSITY OF BIRMINGHAM

SCHOOL OF COMPUTER SCIENCE

COLLEGE OF ENGINEERING AND PHYSICAL SCIENCES

MSC. PROJECT

SDCombo

Semantic Segmentation with Depth Information

Submitted in conformity with the requirements

for the degree of MSc. Computer Science

School of Computer Science

University of Birmingham

Han Li, BSc.

Student ID: 2399859

Supervisor: Dr Eric Finster

September 2023

Abstract

Large models have made great achievements in almost all fields in CV, but it is hard and expensive to train and apply a model with millions of parameters. Soon after a large model succeeded in NLP, it was migrated to computer vision (CV) to solve problems like classification. This wave crashed through almost all fields of CV, and it has been the main path for CV jobs. However, like the NLP, these big models need more and more time and data to train, as well as more and more GPUs. Nowadays, it's common that a model needs tens of GPUs and days to be trained.

Multi-task models may help to improve performance without expanding the scale of parameters. Since the sensors have collected more and more information from the real world, they can help in some jobs besides the basic RGB pictures. This project will focus on finding out whether the performance of semantic segmentation models could be improved with the help of depth information. A multi-task model is built with an overall structure of backbone-segmentation part, depth part, and fusion part-classifier, and tries to find out how to build a model that can benefit from both parts of RGB and depth, and how will the multi-task job affect the performance.

This model can get better performance with the combination of RGB and depth within a reasonable increase of training time and GPU overhead. Some experiments also show that this benefit is not limited to a particular structure, and only a classifier with depth can help improve the whole model's performance as well.

Semantic Segmentation, Cross-modal Task, Multi-modal Task

Acknowledgements

The success of this project would not be possible without the help of many people. I'm extremely grateful to my supervisor Dr Eric Finster and my inspector Michael Oakes for a lot of valuable advice on this project. I am also thankful to my friends that discussing with them helps me in my jobs. Lastly, I'd like to thank my parents for encouraging me when I fall into a dilemma.

MSc. Project

Semantic Segmentation with Depth Information

Han Li Degree Postnomials

Contents

1	Background	1
1.1	Convolutional Neural Network	1
1.2	Multimodal Machine Learning	1
1.3	Multimodal Semantic Segmentation	2
1.4	Class Imbalance	2
1.5	Focal Loss	2
1.6	HHA	2
1.7	Evaluation Metrics for Semantic Segmentation	3
1.8	Encoder-decoder Structure	4
1.9	MobileNet	4
2	Introduction	6
2.1	Project	6
2.2	Report	6
3	Related works	8
3.1	Semantic Segmentation	8
3.2	RGB-D Semantic Segmentation	8
4	Models Structure Design and Selection	10
4.1	Overview	10
4.2	Early Attempts with InternImage	10
4.3	SDHead	11
4.4	SDBottleneck	12
4.5	Experiments	13
4.6	Discovery from Attempts Above	15
4.6.1	Learning Rate	15
4.6.2	Dataset Allocation	15
4.6.3	Stage to Mix the RGB Information and Depth Information	15
4.7	Disadvantages of Model with InternImage	16
4.8	ResNet Fusion	16
4.9	InternImage Fusion	18
5	Models	19
5.1	Overview	19
5.2	Segmentation Part and Depth Part	19
5.3	Joint Part	20
5.4	Data Representation	21

6	Dataset, Experiments and Result	23
6.1	Dataset	23
6.2	Implementation Details	23
6.3	Control Group	23
6.3.1	SDCombo with raw depth	23
6.3.2	MobileNetV3 Large	24
6.4	Results	24
6.4.1	Way to Select the Best Model	24
6.4.2	Evaluation	25
6.4.3	Experiment Results	25
6.5	Analyses	26
6.6	Experiments Analyses	27
7	Conclusion and Discussion	32
7.1	Conclusion	32
7.2	Discussion	32
	References	33
A	Code, Deployment and Weight Files	38

List of Figures

1	IoU	3
2	Confusion Matrix Binary	4
3	Confusion Matrix Multi-class	4
4	Early Attempt	11
5	SDHead	12
6	SDHead for SDBottleneck	12
7	SDBottleneck	13
8	ResNet Fusion	17
9	InternImage Fusion	18
10	Overview	19
11	Segmentation Part and Depth Part	20
12	Joint Part	21
13	Demo of RGB & HHA	22
14	Evulation Accuracy	25
15	Evulation IoU	26
16	Demo	27

List of Tables

1	SDBottleneck-0	14
2	SDHead-0	14
3	SDBottleneck-1	14
4	SDBottleneck-2	14
5	SDBottleneck with SDHead	15
6	Comparing with the control groups	25
7	Classes Evaluation -Accuracy	25
8	Classes Evaluation-IoU	26
9	Performance Comparing	26
10	Experiments Analyses	28

1 Background

This section aims to explain some necessary background works and algorithms that will be used in this project.

1.1 Convolutional Neural Network

Convolutional Neural Network (CNN) is a type of Artificial Neural Networks (ANNs). Because of the strong fitting ability of CNNs, Deep learning has developed rapidly in many fields of CV [44]. CNNs have high freedom for any type of task to customize the neurons, layers, and overall structure so that the model can easily be rebuilt for another task. Thanks to the high degree of freedom, CNN is extremely suitable for multi-modal and cross-modal tasks.

1.2 Multimodal Machine Learning

Our experience of the world is “multimodal”. It is more difficult for humans to distinguish between different objects based on just a photograph than when confronted with a physical object, because humans can combine three-dimensionality in addition to information such as colour and texture.

In machine learning, multi-modal method also works [1]. One example of early multimodal research is audio-visual speech recognition (AVSR) [43], which link the pronunciation with the movement of lips. This is a kind of multimodal between audio and visual, and it shows a positive result in speech recognition performance [28]. Late in 1990s, multimodal approach of visual and word is used in multimedia content analysis such as automatic shot-boundary detection and video summarization [22, 22]. In the early 2000s, this method is applied in understanding human multimodal behaviours during social interactions. Nowadays, the most popular application of multimodal is media description as well as the inverse task media generation from text [2].

There are five main challenges for multimodal machine learning [1]:

- Representation: represent and summarize multimodal data in a proper way.
- Translation: translate (map) data from one modality to another.
- Alignment: find the direct relationship between elements from two or more different modalities.
- Fusion: join information from two or more modalities to perform a prediction.
- Co-learning: transfer knowledge between modalities, their representation, and their predictive models

1.3 Multimodal Semantic Segmentation

The RGB information gives models with distinct colour and texture but not geometric information, so it is difficult to distinguish instances and context which shares the similar colour and texture [46]. Different from the standard RGB-based semantic segmentation, multimodal semantic segmentation employs one or more kinds of data besides RGB pictures to do the segmentation tasks. By introducing geographic information like depth images, multimodal model can get the ability to better distinguish various objects. Also, depth images are potent to environment disturbances such as illumination, fog, etc [36].

1.4 Class Imbalance

Class imbalance is a common problem in dataset for computer vision tasks. Usually, the imbalance happens between the background and target objects or between different class of objects with different frequency in dataset [19, 27].

Training model with an imbalance dataset may make the model to overfit to the training samples from the under-represented classes and may not generalize well during test time [21].

1.5 Focal Loss

Focal Loss [23] is a kind of loss function which is used for training the model on an imbalanced dataset. This loss function is based on the Cross Entropy Loss which is widely used in deep learning of CV.

$$\text{Cross Entropy Loss: } CE(p_t) = -\alpha_t \log(p_t)$$

$$\text{Focal Loss: } FL(p_t) = -\alpha_t(1 - p_t)^\gamma \log(p_t)$$

The focal loss can help the model to be trained on an imbalanced dataset [14].

1.6 HHA

A raw depth map is an image or image channel that contains information relating to the distance of the surfaces of scene objects from a viewpoint. It usually be collected from real world by a Kinect or LiDAR. A raw depth map can capture the shape and size of objects in a scene.

HHA stands for “Horizontal disparity, Height above ground, and Angle with gravity”, it is first defined in [12]. An HHA image can be produced from a raw depth map and corresponding camera metadata [5].

A raw depth map is a direct reflection of distance between camera and surface of objects, while an HHA can give more information about the contours between different objects. For example, a raw depth map may not be able to figure out the contours of the ceiling and the walls,

but it will be an obvious difference in angle with gravity in a HHA image. Also, because the HHA is a 3-channel pictures, it could be treated as a normal picture without extra transforming or encoding.

1.7 Evaluation Metrics for Semantic Segmentation

Semantic segmentation can be treated as a multi-class classification of every pixel in the image. Like the multi-class image classification, accuracy is an important metric for the semantic segmentation models. In semantic segmentation, the accuracy is defined as the ratio of total amount of pixels which are predicted correctly and total amount of pixels of the ground truth [39].

It is a simple metric and most of time it works well, but it can be misleading when the data is imbalanced. For example, if the background pixels are much more than the foreground pixels, a model that predicts everything as background can have a high accuracy.

Comparing to accuracy, Intersection over Union (IoU) is more indicative of success for segmentation tasks compared to accuracy. The IoU is defined as the ratio of total amount of pixels which are predicted correctly and total amount of the pixels which are predicted to be the target or are the target in ground truth. 1

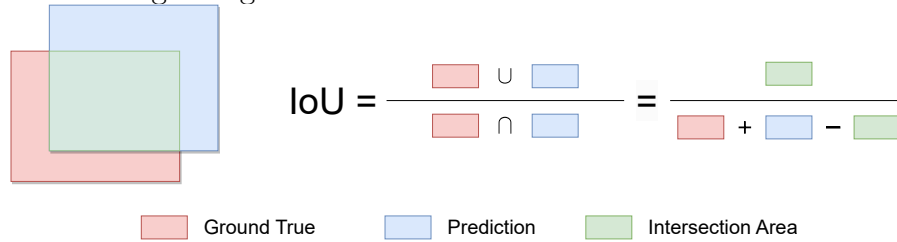


Figure 1: IoU

In practice, accuracy and IoU will be calculated by confusion matrix. A confusion matrix compares the actual target and the result of prediction.

For a binary classification problem, the actual true cases are called “Positive” (P), while the actual false cases are called “Negative” (N). If the predication hit the target (i.e., predict the true cases as “true” or predict the false cases as “false”), it is called “True” (T). Otherwise, it is called “False” (F). A prediction result can be shown as a table 2:

For a multi-class classification problem, the confusion matrix will show the number of cases of each combination of facts and prediction like the table 3:

Accuracy of class i can be calculated as:

$$ACC_i = \frac{TP_i}{G_i}$$

Accuracy of the whole can be calculated as:

	0	1
0	TN	FP
1	FN	TP

Figure 2: Confusion Matrix Binary

		Prection			
		1	2	3	sum
Ground True	1	TP1	FP12	FP13	G1
	2	FP21	TP2	FP23	G2
	3	FP31	FP32	TP3	G3
	sum	P1	P2	P3	T

Figure 3: Confusion Matrix Multi-class

$$ACC = \frac{\text{sum of diagonal}}{\text{sum of all cells}} = \frac{\sum_{i=1}^n TP_i}{T}$$

IoU of class i can be calculated as:

$$IoU_i = \frac{TP_i}{P_i + G_i - TP_i}$$

Mean IoU (mIoU) of the whole can be calculated as:

$$mIoU = \frac{1}{n} \sum_{i=1}^n IoU_i$$

1.8 Encoder-decoder Structure

Encoder-decoder structure is a common structure used in semantic segmentation tasks [25, 29, 31]. It could be divided into two main parts: encoder and decoder. The encoder is used to capture the feature from inputs. The encoder will produce a tensor with a much smaller size, if not only 1, but a much larger channel than the input. This tensor contains the semantic information of the input but in a higher dimensionality. The decoder will recover semantic information from the feature tensor produced by the encoder. Finally, the decoder will produce the result of semantic segmentation which will be a picture with the same size of input. Instead of RGB information, the prediction of each pixel will fill the image.

1.9 MobileNet

MobileNet is a series model introduced by Google. These models show great effectiveness.

MobileNet V1 [17] firstly be proposed in 2017. This model employs depthwise separable convolutions instead of standard convolution, which is the key method to reduce the size and latency.

MobileNet V2 [16] is proposed in 2018. In this version, the depthwise separable convolutions are kept. To improve the performance, they use a “Liner Bottleneck” instead of ReLU6 in the last layer. Also, the bottleneck blocks appear like residual block, so an “inverted residual” is added, which will expand the channel first.

In MobileNet V3 [15] which is proposed in 2019. Besides the use of network architecture

search (NAS) to search for the best model structure, some new methods are used in this model. One is redesigning the expansive layers of the end stage. Also, some ReLU6 functions in deeper layers are replaced with a new function called h-swish.

2 Introduction

Semantic segmentation is one of the most important tasks in computer vision. Since the deep learning succeed in almost every field of computer vision, it also is one of the popular techniques to do the semantic segmentation jobs. Following the works of ResNet, ViT [10], etc., deeper and deeper networks, or called “big models”, are be introduced to get better performance. However, big models need a lot of data, time, and hardware overhead to train and deploy, and the most important, they may performance worse than those small models if they can’t get enough data to be trained on.

Besides the classical RGB-based semantic segmentation, multi-modal models for this task get more and more attention. Some studies show that depth information is a good support for semantic segmentation models to make up some intrinsic weakness of RGB sources, and finally improve the performance of models.

This project aims to build a multi-modal semantic segmentation model with RGB picture and depth information, which can be trained and deployed in a reasonable time and space overhead.

I spend most of the time to build the models and experiments. Only on August and September, I spend more than 550 hours on experiments except coding and evaluation time. Besides the experiments, I spend about 1 week on data pretreatment.

2.1 Project

This project will explore how to make the RGB picture and depth information work together. Below is a brief plan of the project.

- First, a segmentation dataset with both RGB and depth is necessary, and some pretreatment of these pictures will be considered.
- The next step is to build a model that can fuse two types of inputs and get a better result. Some experiments will be taken to test if the model works as expected.
- The last job is the comparison between the new model and existing models.

2.2 Report

In this report, I will use 5 parts to introduce my works:

1. Related Works

This part will introduce some necessary background and some former works related to my project.

2. Models Structure Design and Selection

Here are some primary attempts before building the model. These attempts try different structures to make the RGB images work with depth information. Some ideas of the final model come from here as well.

3. Model

This part introduces the main model of the project, with a detailed description of each part and data representation.

4. Dataset, Experiments and Result

This part includes the selection of dataset, pretreatment, implementation of the experiments and the setting of control groups. Experiments result and some brief analyses are included as well.

5. Conclusion and Discussion

The conclusion of this project and some issues need to be discussed.

3 Related works

3.1 Semantic Segmentation

Semantic segmentation is one of the main tasks in the field of computer vision, and deep learning has been the main method to solve these problems [11, 42]. [25] succeeded at converting the classification network to a pixel-wise segmentation network, replacing the fully connected layers with convolutional layers. This also started a new age of semantic segmentation with the approach “Fully Convolutional Network” (FCN).

Though the FCN model is powerful and flexible for most tasks in computer visions, but many problems remain: lack of global context information, no instance-awareness by default, efficiency far from real-time execution at high resolutions, and suitability for unstructured data such as 3D point clouds or models. To solve these problems, many methods are introduced. For example, Encoder-decoder [25, 29, 31], Conditional Random Field (CRF) [3, 34], Dilated Convolutions [4, 41], Multi-scale Prediction [30, 32], Feature Fusion [24], Recurrent Neural Networks [35] and so on.

With the success of ResNet [13] and its residual blocks, the models break the limit of depth. To get more details and the whole scene at the same time, more and more models have a large parameter scale. However, it also means more and more time and data to train. With small data and a brief time to train, these big models may perform worse than the small ones.

3.2 RGB-D Semantic Segmentation

Semantic segmentation with RGB images faces many challenges such as shadow, mirror, and similar texture on neighbours, and other information can highlight the difference between objects in other degrees [37]. Due to the development of Autonomous Vehicles, more and more depth information is collected, and making a model with both RGB pictures and depth information is highly practised. Depth measurement can help identify the boundaries of objects and offer geometric information of dense scene elements, which is a helpful support to semantic segmentation in some aspect like edge detection [20].

Due to the different formats of RGB images and depth information, they need two methods to unified format features and fuse them for segmentation. Most multi-modal semantic segmentation methods are divided into two categories: one will mix the RGB and depth in the input stage, then use the mixed tensor as a normal segmentation input to extract feature [6, 18]. The other one will extract features of RGB and depth separately, then use the two features to get the final feature tensor for classifier [9].

In [45], a new model is introduced which is called CMX. As the key point of this model,

a new method is proposed that the features from two pipelines can interact with each other. They design a two-branch structure to extract feature from RGB and depth separately and interact with each other in a joint stage. The point is that the joint stage will produce the cross-modal feature for next RGB stage and depth stage to further deepen and improve the feature extraction. In this framework, two branches can rectify each other. In CMX, they employ Mix Transformer encoder (MiT) [33] pretrained on ImageNet as the backbone. They design a Cross-Modal Feature Rectification Module (CM-FRM) to rectify one feature regarding another feature and a two-stage Feature Fusion Module (FFM) to fuse features belonging to the same level into a single feature map with the cross-attention mechanism.

4 Models Structure Design and Selection

4.1 Overview

There are several ways to build a model for semantic segmentation, and each one has its advantages and disadvantages. Also, some of these models are easy or suitable to transform to a multi-modal model for RGB-D semantic segmentation. In this section, I will introduce some model with different structures and its performance with a quick experiment and evaluation. After comparing with each other, the best model will be selected to carry on the detailed experiments.

During the searching for a better model, I tried different backbones and structures. These backbones include InternImage [38], ResNet [13] and MobileNet [15, 16, 17].

4.2 Early Attempts with InternImage

The main idea of the early model structure is to combine the depth information with a complete result of a semantic segmentation. In more detail, I employ a complete semantic segmentation model to produce the semantic segmentation prediction, then add the depth information to get a better result.

To capture enough information from RGB images, I employ a state-of-the-art model for semantic segmentation tasks. By the time of this report writing, InternImage is still the state-of-the-art model in CityScapes [8] and Pascal Context [26]. InternImage produces four tensors from different scales and employs a UPerNet [40] network as a classifier.

I build a simple module to pretreat the depth information to align with the semantic segmentation result and combine the depth tensor and semantic segmentation tensor to produce a mixed feature. The semantic segmentation prediction will be produced from the mixed feature.

I spend about 2 months on this part. I spend more than 250 hours on experiments, expect time on coding. These early attempts are almost discarded but many experiences on learning rate, data augmentation and structure designing are helpful on the next works of the final models.

The first try has a lot of problems:

- The original model is based on MMLab [7]. It's easy to train and deploy, but difficult to change some module of the model.
- The model with depth information doesn't have much difference with the normal model, sometimes even worse. This may be due to the mix module is too small for the backbone.

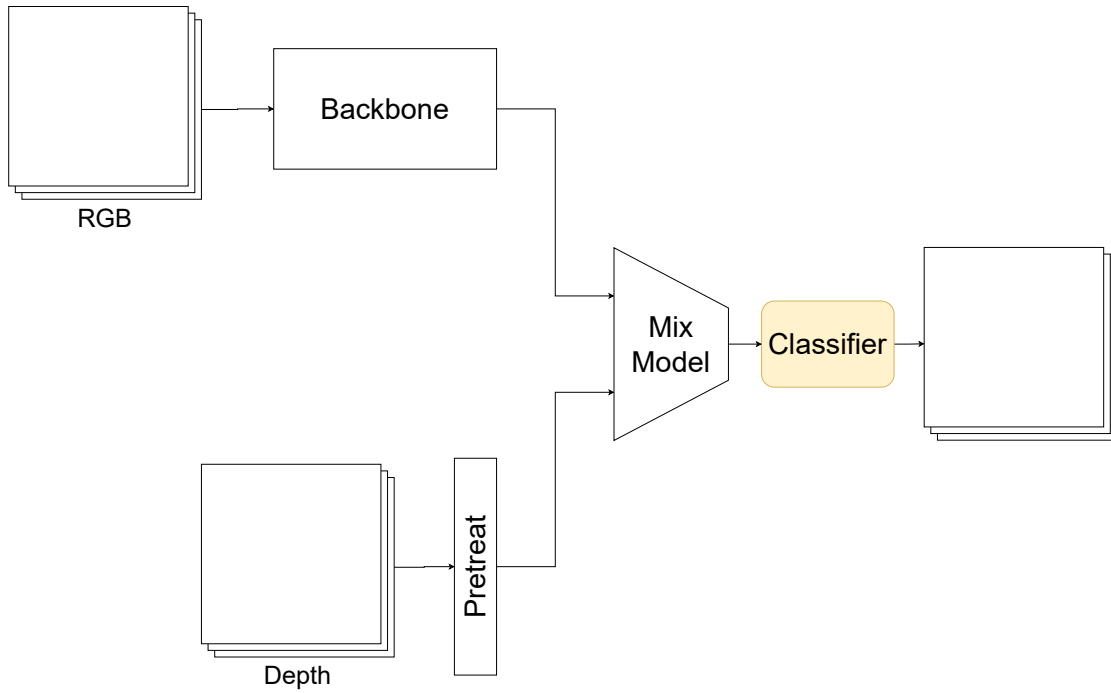


Figure 4: Early Attempt

Also, the mix module is the end of the model, and it's hard for loss function to backward to the whole model layers.

To make the model works more properly, I made some preparation:

- First, rewrite the whole model of InternImage in PyTorch. By remove all MMLab modules, this model can have more flexibility to for depth information. I spend about 1 week to do this job.
- Also, UPerNet is also not necessary, and even make the loss more difficult to adjust the deeper layer.
- After the rewriting of the model, I build two new modules: SDHead and SDBottleneck. The former one is used as a classifier and the later one is used as a special module to mix the RGB tensors and depth information.

4.3 SDHead

The SDHead is a simple module to mix the RGB tensors and depth information. The inputs of this module are four tensors from InternImage and one tensor from depth information. The depth tensor will be scaled to the same size as each RGB tensor respectively, concatenated to corresponding RGB tensor, and mixed by a convolution layer to learn the mixed feature. Like

a Feature Pyramid Network (FPN), these four tensors will be added together to produce the prediction tensor.

To cooperate with the SDBottleneck later, I decide another SDHead to accept 4 feature tensor from different scale, RGB tensor and depth tensor.

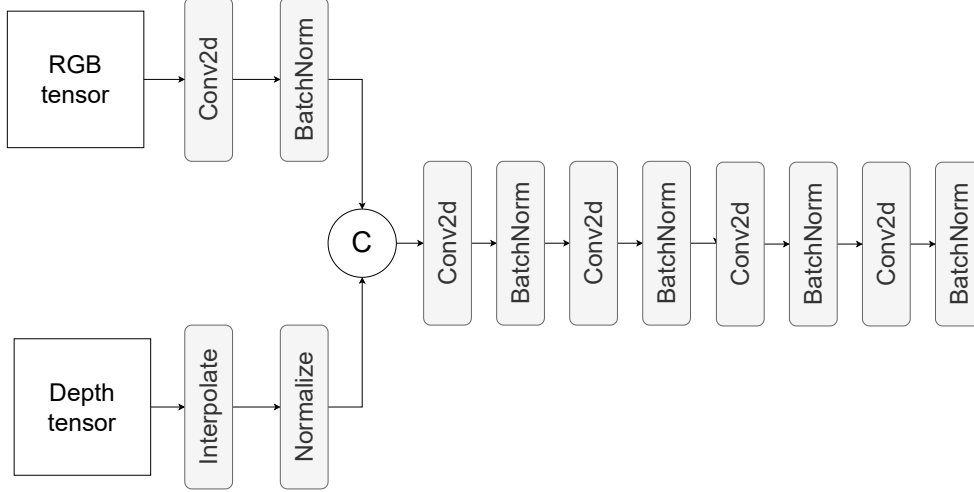


Figure 5: SDHead

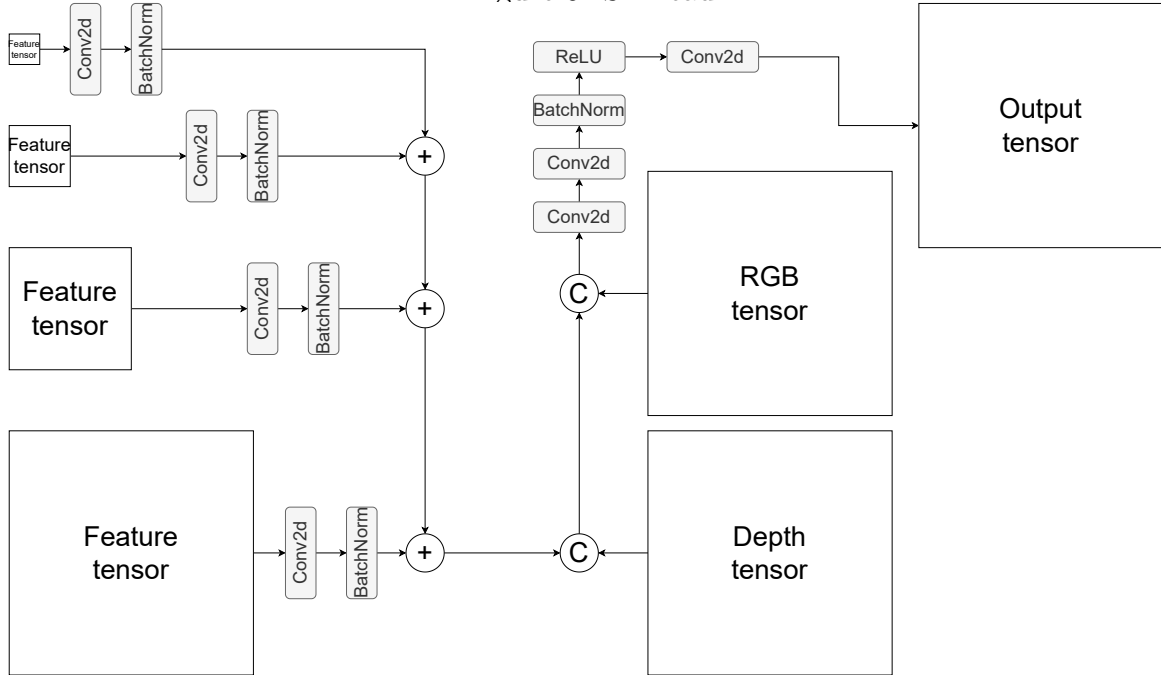


Figure 6: SDHead for SDBottleneck

4.4 SDBottleneck

The SDBottleneck is a module between the backbone and the classifier to make fusion of RGB and depth. In this module, the inputs are four tensors from backbone and one tensor from depth information. The depth tensor will first be normalized to $[0,256]$, then concatenate to

RGB tensors respectively. To align with the size of RGB tensors, a downsampling will be taken by a convention layer (kernel size=3, stride=2). These mixed tensors will be learned by a 4-module convolution module. Also, the original RGB tensors will be sent into a FPN-like network to produce a RGB feature tensor. Finally, the depth tensor, the RGB feature tensor and four mix tensors will be sent to classifier.

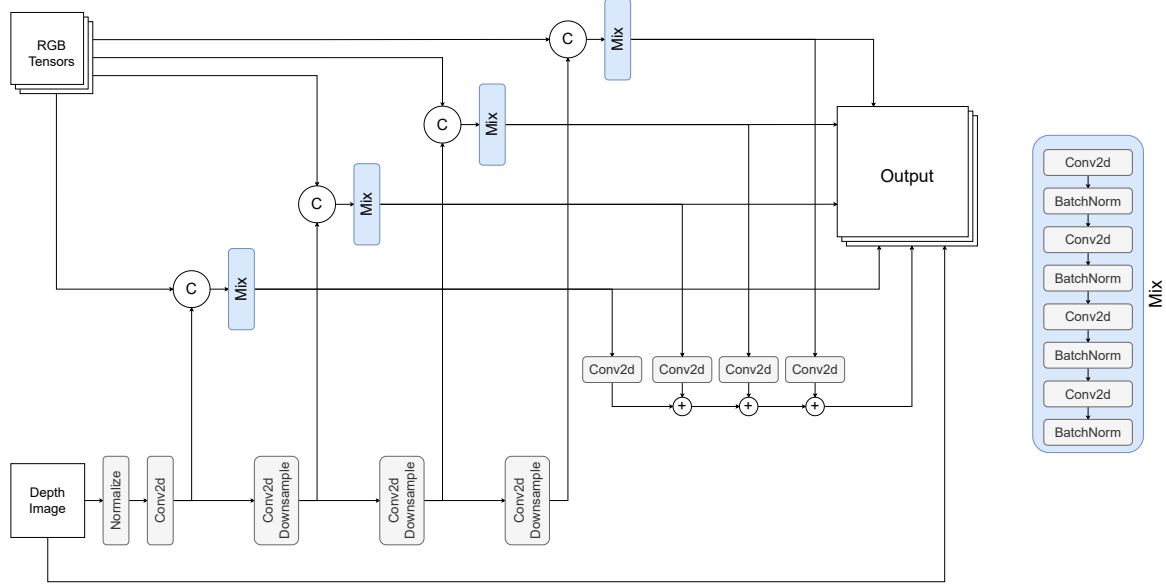


Figure 7: SDBottleneck

4.5 Experiments

To test the performance of different models, I spend more than 200 hours on the experiments. Here are some typical experiments to show some important conclusion.

Dataset: Stanford2D3D

Dataset setting:

Set 1 (Whole):

Training set: areas [1, 2, 3, 4, 6]

Valuation set: areas [5]

Set 4 (Small):

Training set: areas [4]

Valuation set: areas [3]

crop size: 256^2

Pre-trained weight: trained on ADE20K with the crop size of 512^2 for 160k iterations.

Unless otherwise stated, the classifier of SDBottleneck is UPerNet.

Table 1: **SDBottleneck-0**

simply mix the semantic segmentation and depth information

Dataset	Initial lr	Accuracy	mIoU	Details	Training Time
Whole	0.0001	65.1	36.4	The loss shocks	14:05:24
Whole	5.00E-05	70.4	43.8		12:35:24
Small	5.00E-05	66.9	40.7		3:50:33
Small	5.00E-05	68.4	40.3	without depth	3:37:56

without pretrained

Dataset	Initial lr	Accuracy	mIoU	Details	Training Time
Whole	0.001	52.9	19.8	The loss shocks	13:36:28

Table 2: **SDHead-0**

simply mix the semantic segmentation and depth information

Dataset	Initial lr	Accuracy	mIoU	Details	Training Time
Small	5.00E-05	68.4	40.3	batch size = 8	3:37:56
Small	5.00E-05	63.6	32.7	batch size = 4	5:56:45

Table 3: **SDBottleneck-1**

Add some convolution layers and introduce Focal Loss

Dataset	Initial lr	Accuracy	mIoU	Details	Training Time
Small	5.00E-05	68.1	40.9	without Focal Loss	3:51:10
Whole	5.00E-05	67.8	41.6	without Focal Loss	11:39:05
Whole	5.00E-05	67.2	41.6	introduce Focal Loss	15:35:24
Whole	5.00E-05	64.7	35.8	crop size=125	8:57:21
Small	5.00E-05	69.4	43.7	crop size=256	3:44:32

Table 4: **SDBottleneck-2**

Make some tunings on depth part

Dataset	Initial lr	Accuracy	mIoU	Details	Training Time
Small	5.00E-05	69.9	44.1	adaptive average pooling	3:51:48
Small	5.00E-05	70.1	44.4	a convolution layer as downsample	3:55:52
Small	5.00E-05	73.9	47.9	crop size=512	13:19:55

Table 5: **SDBottleneck with SDHead**

Use SDHead instead of UperNet					
Dataset	Initial lr	Accuracy	mIoU	Details	Training Time
Whole	5.00E-05	67.1	39.8	batch size=batch size/2	7:55:08
Whole	5.00E-05	66.8	38.5	batch size=1	9:47:21
Small	5.00E-05	68.2	42	use cross entropy	2:02:35
Small	5.00E-05	69.1	42.7	without depth	1:53:23
Whole	5.00E-05	66.4	39.6	Focal Loss	7:57:34

4.6 Discovery from Attempts Above

From the primary attempt above, there are some useful information needed to be record, which will the basic rules for later models and experiment.

4.6.1 Learning Rate

When the learning rate is set above $1e-4$, the loss of the model will shock and cannot converge in reasonable epochs. When the learning rate decreases to low than $1e-7$, the loss will decrease too slow. So, the learning rate is set to $[5e-5, 1e-7]$ to make the model coverage in reasonable epochs.

4.6.2 Dataset Allocation

To exam the effect of some small changes to the model, I employ a small dataset with area 4 as training set and area 3 as test set. After some experiments, it shows that on this small dataset, models can get a better evaluation result than on the whole dataset, but models with better performance on the small dataset show better performance on the whole dataset as well. So, this small dataset can be used to quick check for models. Also, some experiments on less data can also be taken on this setting.

4.6.3 Stage to Mix the RGB Information and Depth Information

To find out how different stages will affect the performance of the multi-modal model, I tried different combinations of backbone, SDBottleneck, SDHead and classifier.

1. At the Final Stage

The experiments of SDHead show that mix the semantic segmentation at the end of the model will get a worse result even than original model. There are some possible reasons:

One of these reasons is the size of model. The fusion part is much simpler than the backbone and classifier, and this bring more instability than robustness to the model. In

many experiments, the model often gets the best result not in the last epochs.

Another reason is that the depth information is added into the model in the surface part with the almost raw information. It's hard for the model to learn and extract enough information from the depth information and the mixed tensors.

2. Between backbone and classifier

Using a SDBottleneck to introduce the depth information in the middle of the model show better performance than SDHead. The SDBottleneck is much more complex than SDHead and it mixes the depth information with the more original RGB features comparing with SDHead.

4.7 Disadvantages of Model with InternImage

InternImage is a good backbone for a computer vision model, and it's also easy to employ it in a multi-modal model. But it's a big model that not easy to train it or to do some quick experiments on it. Each experiment with it will take more than 4 hours and even evaluation need to spend more than half a hour. This makes it really difficult to fine tune the models quickly.

Also, it's too hard to split the model to change the deeper modules. Most changes on shallow layers can't make any sense because them are too small for the model itself, but most changes on deeper layers make it worse on performance.

4.8 ResNet Fusion

As said in the former section, the depth information can work better if it is added into the model in a deeper layer. To make a more flexible and a more lightweight model, I employ ResNet as the backbone to build a new model. The ResNet is more flexible than InternImage and easier to be divided to add more modules in deeper layers. This new model will be called as ResNet Fusion.

In the ResNet Fusion, I split the model to 4 stages. Between each stage, I add a fusion module to mix the depth information with the RGB feature and these modules produce mixed features in different scales. All 4 mixed features are sent to a simple classifier which has an FPN-like module to produce the result from these features.

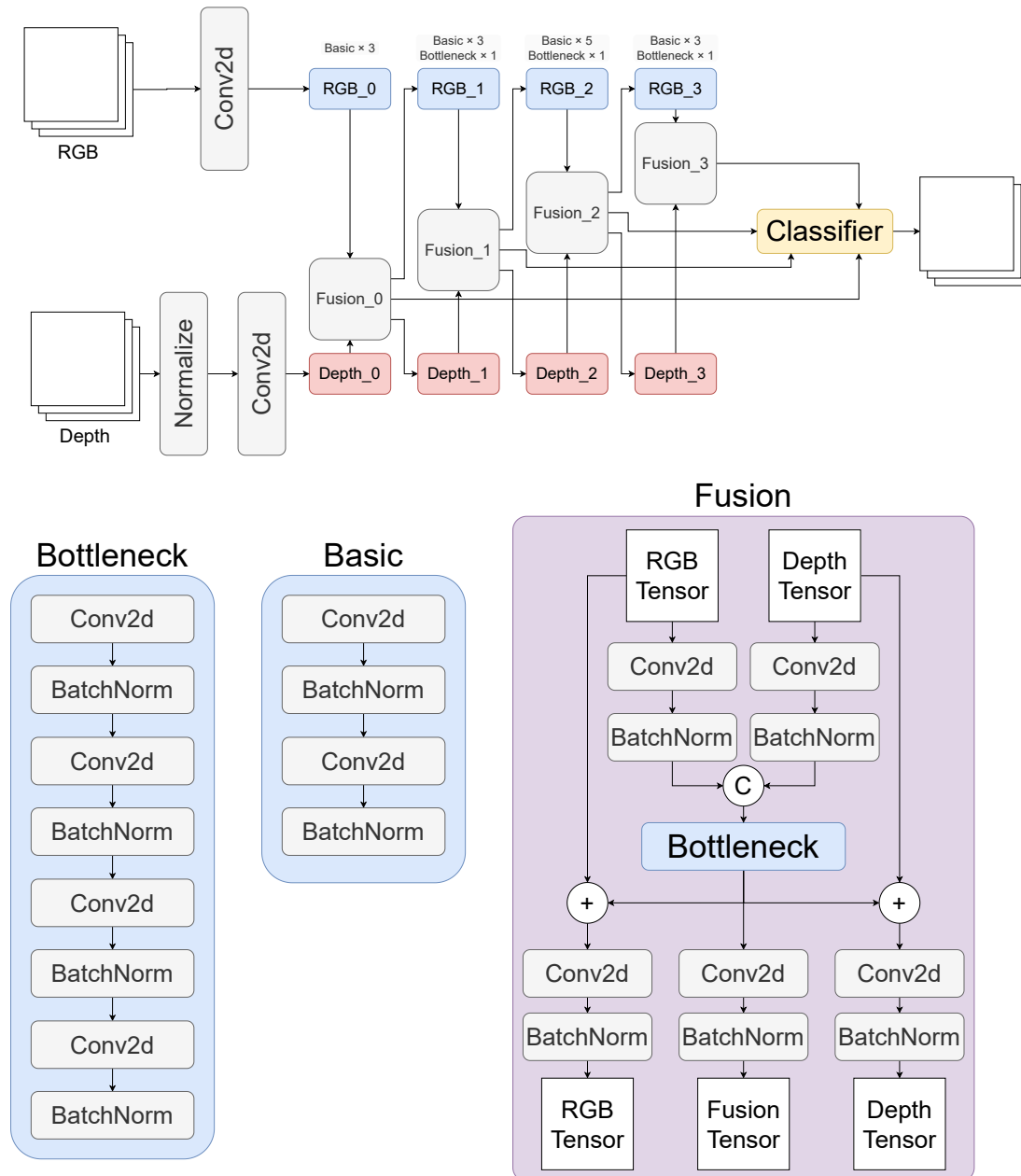


Figure 8: ResNet Fusion

4.9 InternImage Fusion

Like the ResNet Fusion, this model uses a fusion part to mix the two images but make some changes to suit the InternImage output.

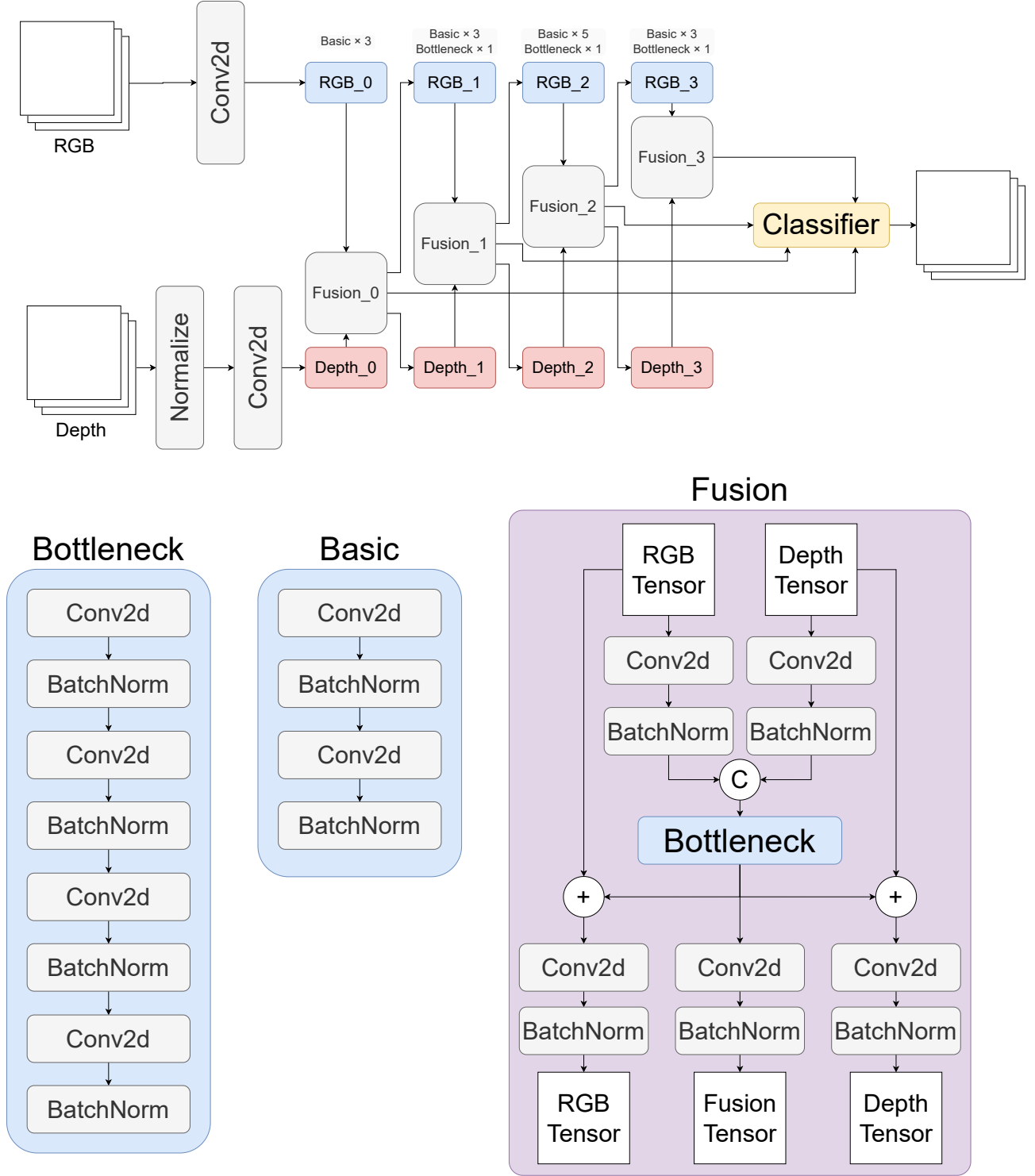


Figure 9: InternImage Fusion

5 Models

5.1 Overview

The model mainly contains three parts: segmentation part, depth part and joint part. Since the RGB images and depth images have different encoding, I employ a two-branch architecture to extract features from RGB and depth parallelly. This two-branch architecture could give the model the ability to capture the unique characteristics of the respective input. The two branches will share a same pipeline of changes to channels and sizes, which make sure that they are aligned and ready for the interaction.

During the pipeline, these two parts will interact with each other by the joint part. The joint part will be a junction for two pipeline to exchange information and rectify each other. The inputs and corresponding outputs will keep in same size and channel to connect to the next stage.

In this project, I apply the MobileNetV3 Large as the backbone. In the joint part, I employ a ResNet bottleneck block to make the fusion of two basic features. The tensor out from the last joint part will be the final feature for semantic segmentation, with DeepLabV3 as the classifier.

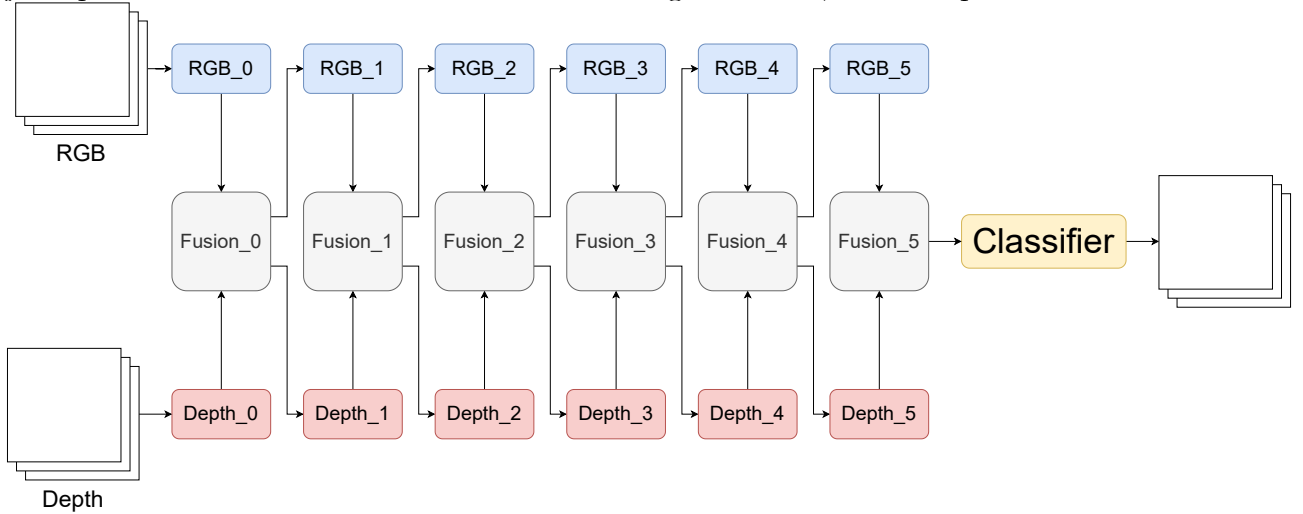


Figure 10: Overview

5.2 Segmentation Part and Depth Part

The segmentation part and depth part depend on MobileNetV3.

According to the changes of feature size, the MobileNetV3 Large can be divided into 6 stages. In this model these stages are devised, and a fusion stage is added between these stages. Except the first stage whose input is the origin RGB images and depth images, the input of each stage is produced by former fusion stage and output is the input of next fusion

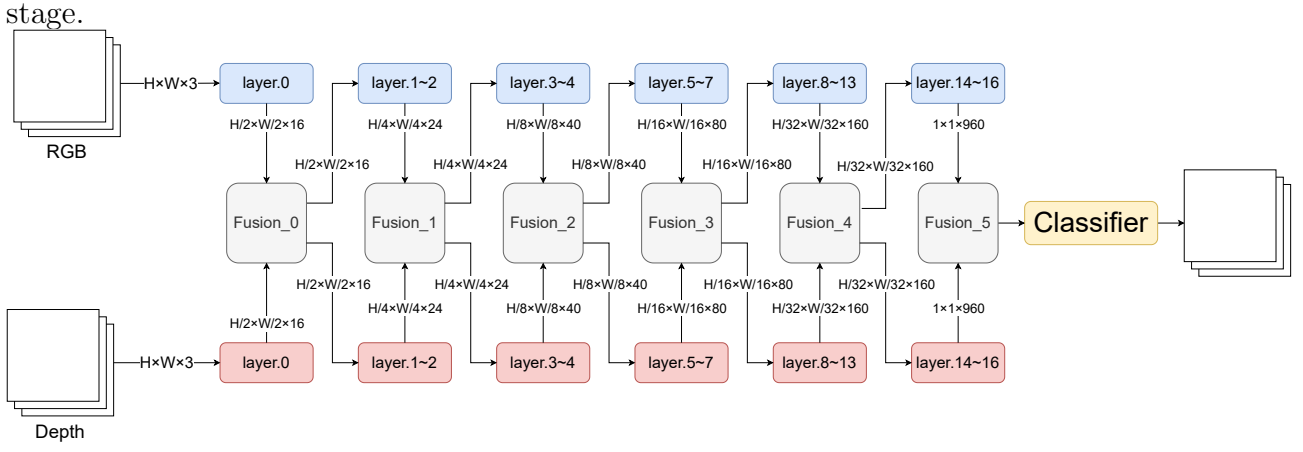


Figure 11: Segmentation Part and Depth Part

5.3 Joint Part

The joint part is a series of fusion modules between segmentation and depth stages.

Each Fusion module will take two inputs from segmentation part and depth part separately. A pretreat convolution layer will apply on them to cut the channel to the half, then concatenate them as a whole tensor with the same channel as the input. This “joint” tensor will be sent to two ResNet bottleneck blocks to get the fused RGB and fused depth tensor respectively. The fused RGB and fused depth tensor will be cut half channel again and concatenate with the opposite former half channel tensor. In other words, the fused RGB tensor will be concatenated with the half channel depth tensor, vice versa. The “hybrid” tensor will be sent to two ResNet bottleneck blocks again to get RGB result and depth result. The Fusion module will produce three outputs: RGB, depth and fusion. The RGB and depth output is the RGB result, and depth result add with corresponding origin input, and the fusion output is from the addition of RGB result and depth result.

The RGB and depth outputs will be the inputs of next stages of segmentation part and depth part’s input respectively. The fusion output can be used as the feature in this scale for classification. In this model, only the fusion output of the last fusion stage will be used as the final feature for classifier to produce the segmentation result.

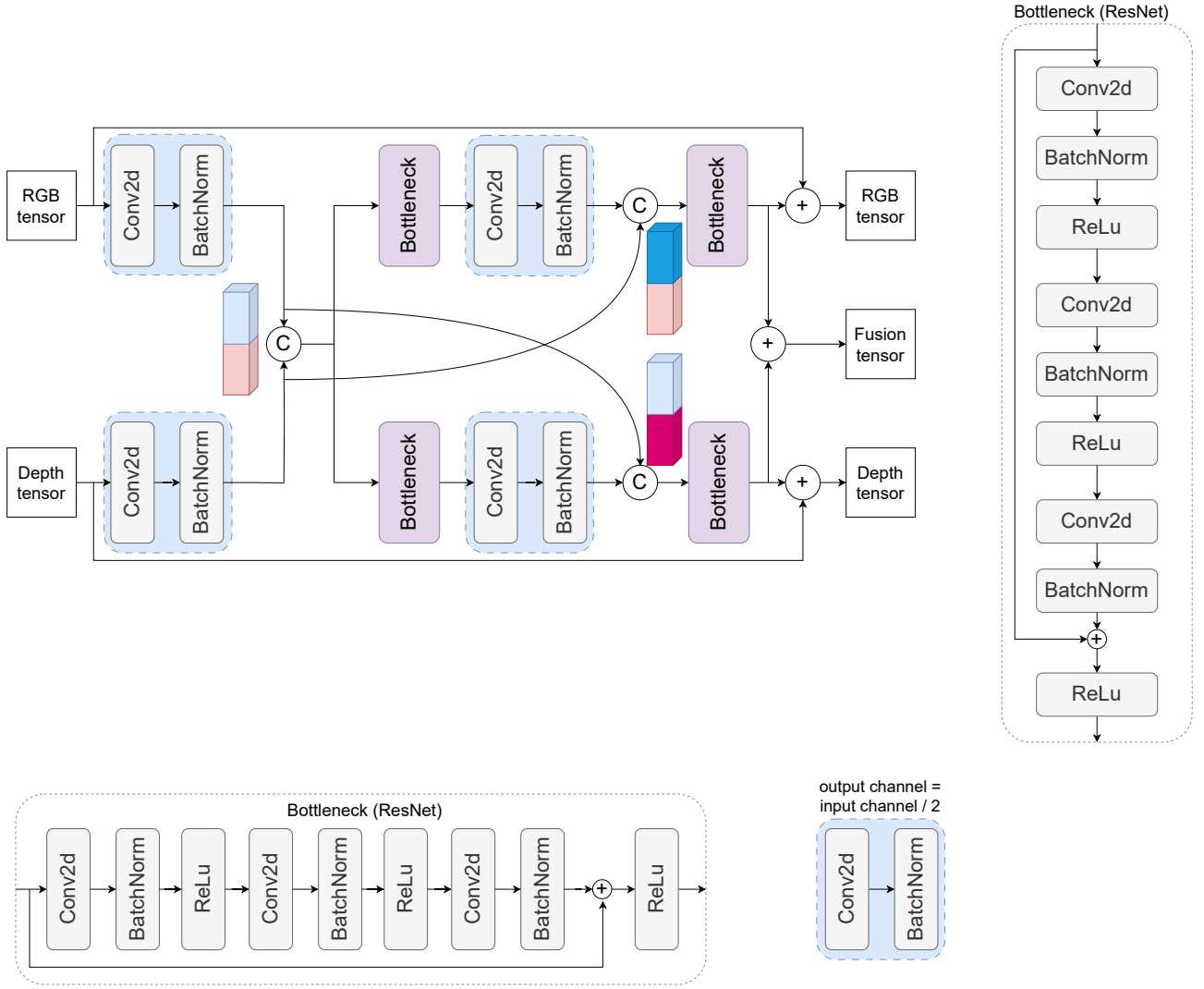


Figure 12: Joint Part

5.4 Data Representation

The depth images can offer the information of position and contour naturally. In this project, I encode the depth images into HHA format. The HHA can offer the geometric properties, including horizontal disparity, height above ground, and angle with gravity. Comparing to the raw depth images with only the distance between camera and corresponding point on real world, HHA can represent geometric information more directly with three channels. Also, the format of 3 channels is the same with the RGB images, so that it is easier for existing models to treat the depth images. Some methods which are helpful to improve the performance and efficiency but designed for RGB images can also be used in depth images.



Figure 13: Demo of RGB & HHA

6 Dataset, Experiments and Result

6.1 Dataset

Stanford2D3D is a dataset of large-scale indoor spaces that provides a variety of mutually registered modalities from 2D, 2.5D and 3D domains, with instance-level semantic and geometric annotations. This dataset contains a total of 70,496 regular RGB along with their corresponding depths, semantic annotations, and camera metadata. The depth images are saved as 16-bit grayscale PNGs where one unit of change in pixel intensity corresponds to a $\frac{1}{512}$ m change in depth. The maximum observable range is therefore about 128 meters ($2^{16} \cdot \frac{1}{512}$). All depths beyond this maximum distance assume the maximum value (65,535).

Before the implementation of experiments, I encode the raw depth images to HHA format. As said in Background, HHA can give geometric information more directly, and it's easier to be treated by the MobileNet.

6.2 Implementation Details

Each picture is performed by random flipping with a probability of 50%, scaling with random scales [256, 1080] and will be cropped into [256, 256]. Random flipping and scale could help improve the performance of a model by exposing it to additional variations of the same images. The depth and ground truth of semantic segmentation will be treated with the same processes and settings to keep them aligned with each other.

Areas of 1, 2, 3, 4, 6 are used for training and area 5 is for testing. The batch size is set on 30.

I employ the AdamW as optimizer, with weight decay=0.01, betas= (0.9, 0.999).

The original learning rate is set as 5×10^{-5} , and I employ a Cosine Annealing learning rate schedule with the minimum learning rate= 10^{-7} . The maximum number of iterations is set as the same as the epochs of training. In another word, I use the Cosine Annealing learning rate schedule as a Linear learning rate schedule.

I use Focal Loss as the loss function with alpha=0.5, gamma=2.

6.3 Control Group

To show the performance improvement intuitively, some control groups are set for comparison.

6.3.1 SDCombo with raw depth

This control group is set to show the difference of performance between models with raw depth images and HHA images.

In this experiment, the model will apply the same training setting with the main model. The only difference with the HHA version is the pretreat of depth images. The raw depth images have only one channel with the range of $[0, 63353]$, while the HHA images have the same data range and format with the RGB images. So, a data pretreat is employed to match the input format of the model.

```
depth = nn.functional.normalize(depth.type(torch.float)) * 256
depth = self.depth_conv(depth.unsqueeze(1))
```

First, the raw depth images are normalized to $[0, 256]$. Then an extra axis is added to match the format of input of the model.

6.3.2 MobileNetV3 Large

This control group is set to show the difference of training efficiency between RGB-based model and RGB-D model.

In this experiment, due to the different size between MobileNet and SDCombo, some changes are applied to make the MobileNet could get the best performance. To make sure the MobileNet could be trained in the most efficient way, the batch size is adjusted to 26. Also, MobileNet is much smaller than SDCombo, so the training epochs is set on 20 epochs to match the training time of SDCombo. Other settings are as the same as the SDCombo.

6.4 Results

6.4.1 Way to Select the Best Model

During the training, I apply a fast evaluation after each epoch of training. This fast evaluation applies almost the same data treatment pipeline with training but without random scaling and flip. The input images will be cut to $[256, 256]$ randomly. The batch size is set by the expression:

$$batch_size = \max\left(\frac{2^{18}}{crop_size^2}, 1\right)$$

This batch size is the most effective way to operate under the limitation of the training machine (GeForce RTX 2060 6GB).

The evaluation will calculate the confusion matrix of the predictions and then produce out accuracy and IoU for each class and the whole from confusion matrix.

Each evaluation result will be recorded and the model with the best mIoU will be selected as the best model of this training. If two or more model have the same mIoU, the best model will be the one with higher accuracy. If the accuracy still can't tell the difference, check and compare the classes' IoUs from the lowest one to the highest until find the better or best one.

6.4.2 Evaluation

Evaluation will be taken on the “area 5” of the dataset.

The data pipeline is similar to the fast evaluation during the training, some differences include that the crop size is set on the full size of the picture, [1080, 1080]. The batch size is 1 due to the limitation of GPU memory.

A stand-alone script will make the formal evaluation for the best model of each training. The weight file will be loaded first, and the model will be created according to the model parameters and metadata in weight file.

6.4.3 Experiment Results

Table 6: Comparing with the control groups

Model	Training Epochs	Training Time	mAcc	mIoU
DeepLabV3	20	8:33:49	62.8	35.7
SDCombo with Depth	10	6:50:36	45.5	17.6
SDCombo with HHA	10	6:03:53	65.4	36.5

Table 7: Classes Evaluation -Accuracy

Accuracy	beam	board	bookcase	ceiling	chair	clutter	column
DeepLabV3	6.8	58.2	64.1	73.6	56.6	54.2	6.1
Depth	0.4	11.2	36.9	28.1	10.6	65.6	6.8
HHA	41.4	45.7	49.9	83.3	46.4	54.2	9.7
Accuracy	door	floor	sofa	table	wall	window	
DeepLabV3	16.7	69.7	20.3	56.9	84.3	35.9	
Depth	22	14.1	0.3	31.8	70.4	42.8	
HHA	22.5	85.4	4.3	74.2	87.8	43.8	

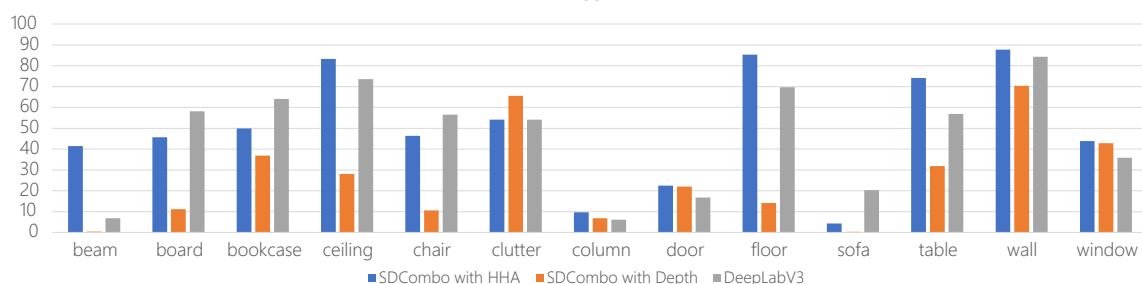


Figure 14: Evulation Accuracy

Table 8: **Classes Evaluation-IoU**

IoU	beam	board	bookcase	ceiling	chair	clutter	column
DeepLabV3	0.5	42.1	44.1	65.5	41.9	28.8	4.5
Depth	0.1	11	29.1	25.6	9.5	20.8	3
HHA	1.5	36.2	41.9	72	35.2	31.5	6.2
IoU	door	floor	sofa	table	wall	window	
DeepLabV3	12.9	66.1	18.1	45	60.2	34.5	
Depth	14.2	13.9	0.3	24.2	47.7	28.9	
HHA	18.3	80.3	3.6	45.2	62.1	40.9	

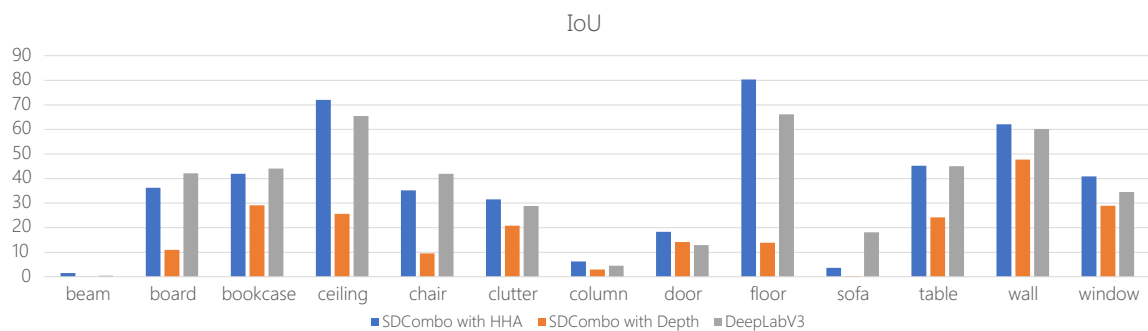


Figure 15: Evulation IoU

Table 9: **Performance Comparing**

Dataset	Training Epochs	Training Time	Crop Size	mAcc	mIoU
Whole	10	6:03:53	256	65.4	36.5
Whole	10	15:42:45	512	72.5	47.2
Small	10	1:15:14	256	65.8	38.8
Small	40	6:50:40	256	62.6	45.6

6.5 Analyses

From the result above, it's obvious that the SDCombo has better performance than the original RGB-based model with less training time and epochs. This shows that depth information can help a multi-modal model to do the semantic segmentation jobs better than only using RGB images.

Comparing between SDCombo with HHA and with raw depth image, depth information encoding is an important factor in a muti-modal model's performance.

The experiment on a bigger crop size shows that bigger sampling can help the model to learning the features more efficiently. This model's performance can still be improved with

more training.

The experiment on a smaller dataset with more training epochs shows that SDCombo can get a good performance training on a small dataset.

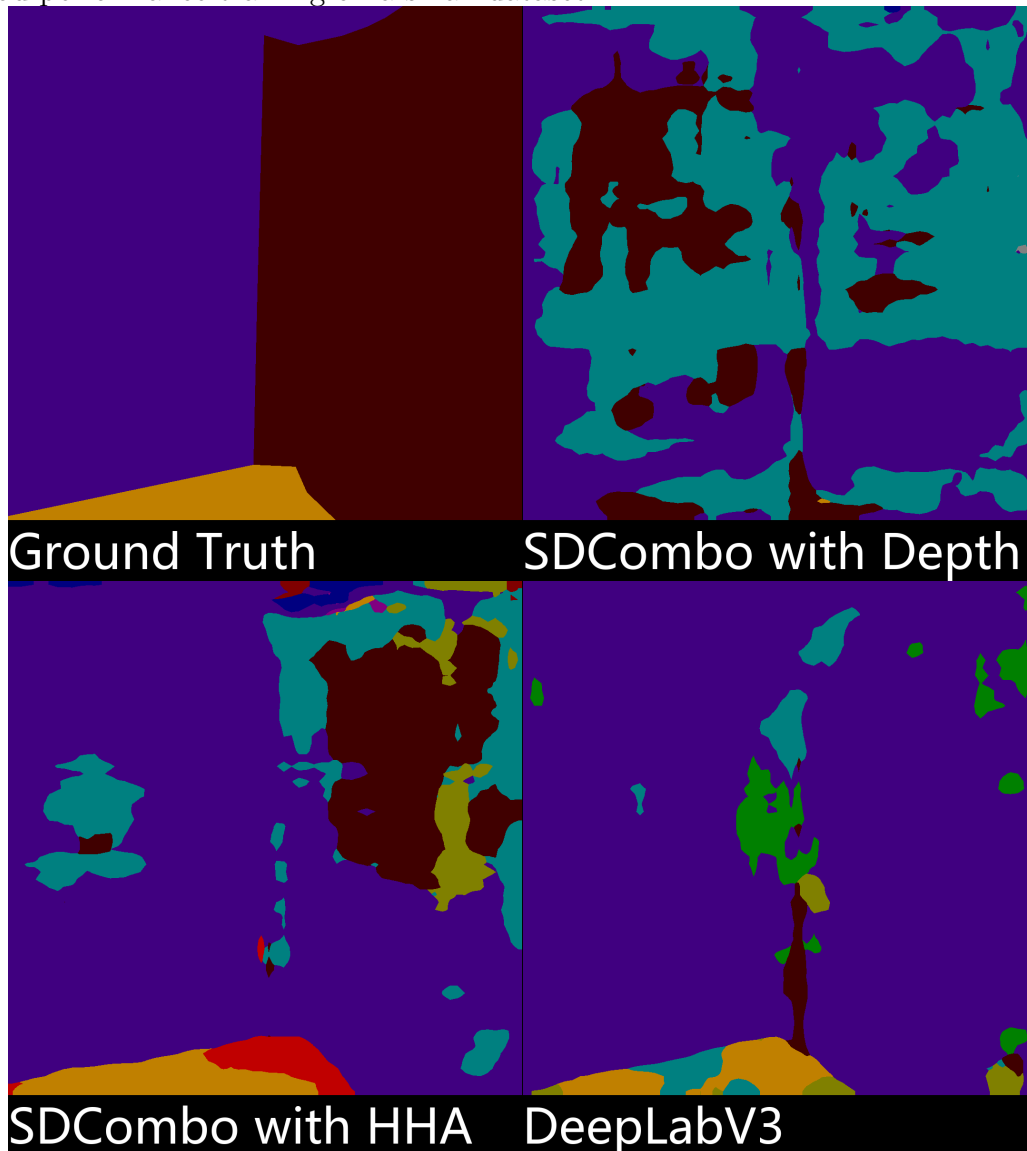


Figure 16: Demo

Sometimes depth information makes the model so sensitive that it can't treat the large object well. The model may tend to divide a large object into small objects rather than treat it as a whole.

Some objects have similar depth features but look really different in RGB pictures (e.g. a table and the floor).

6.6 Experiments Analyses

Here is the experiment logs from 2023.07.28. Experiments before this day are not recorded. Some of these experiments focus on testing the backbones.

You can also find more details of experiments below in folder "work_dir" in the git. The

raw records include the log of each step, model information and training time.

Table 10: **Experiments Analyses**

Marks	Date	Start Time	End Time	Total Time
20230728_150833	07/28	15:08:50	15:10:09	0:01:19
20230728_152434	07/28	15:24:52	15:25:09	0:00:17
20230728_160802	07/28	16:08:31	16:17:37	0:09:06
20230728_162147	07/28	16:22:08	17:02:30	0:40:22
20230728_211711	07/28	21:17:32	5:17:34	8:00:02
20230730_074226	07/30	7:42:49	13:09:01	5:26:12
20230730_131421	07/30	13:14:38	16:27:48	3:13:10
20230731_105559	07/31	10:56:15	10:56:38	0:00:23
20230731_110122	07/31	11:01:37	12:42:46	1:41:09
20230802_111329	08/02	11:13:46	11:48:09	0:34:23
20230802_130450	08/02	13:05:14	15:52:49	2:47:35
20230802_160146	08/02	16:02:02	16:45:55	0:43:53
20230802_164633	08/02	16:46:49	17:00:14	0:13:25
20230802_170457	08/02	17:05:11	19:20:26	2:15:15
20230802_204358	08/02	20:44:23	21:18:35	0:34:12
20230803_181329	08/03	18:13:52	18:41:42	0:27:50
20230803_190110	08/03	19:01:33	20:42:51	1:41:18
20230803_204453	08/03	20:45:21	10:21:49	13:36:28
20230804_123637	08/04	12:37:08	15:14:11	2:37:03
20230804_154655	08/04	15:47:23	16:01:39	0:14:16
20230804_163004	08/04	16:30:30	16:44:45	0:14:15
20230804_164734	08/04	16:47:59	17:05:48	0:17:49
20230804_204744	08/04	20:48:19	22:22:15	1:33:56
20230804_222235	08/04	22:23:07	12:28:31	14:05:24
20230805_154709	08/05	15:47:35	18:56:05	3:08:30
20230805_202214	08/05	20:22:46	8:58:10	12:35:24
20230806_090018	08/06	9:00:53	12:51:26	3:50:33
20230806_134041	08/06	13:41:11	17:19:07	3:37:56
20230806_181949	08/06	18:20:07	19:48:45	1:28:38

20230806_211145	08/06	21:12:03	3:08:48	5:56:45
20230807_164017	08/07	16:40:48	20:31:58	3:51:10
20230807_204708	08/07	20:47:55	8:27:00	11:39:05
20230808_085650	08/08	8:57:19	12:38:09	3:40:50
20230808_131252	08/08	13:13:22	17:53:34	4:40:12
20230808_203524	08/08	20:36:00	20:57:27	0:21:27
20230808_205841	08/08	20:59:12	11:54:44	14:55:32
20230809_120044	08/09	12:01:11	17:23:17	5:22:06
20230809_202343	08/09	20:24:39	20:29:11	0:04:32
20230809_205650	08/09	20:57:27	12:32:51	15:35:24
20230810_130727	08/10	13:07:58	18:26:46	5:18:48
20230810_200628	08/10	20:07:02	20:25:13	0:18:11
20230810_211350	08/10	21:14:23	21:23:32	0:09:09
20230810_212534	08/10	21:26:07	21:32:25	0:06:18
20230810_213258	08/10	21:33:35	6:30:56	8:57:21
20230811_064658	08/11	6:47:27	10:31:59	3:44:32
20230811_105109	08/11	10:51:37	11:13:21	0:21:44
20230811_112905	08/11	11:29:39	15:21:27	3:51:48
20230811_153904	08/11	15:39:31	19:35:23	3:55:52
20230811_200817	08/11	20:08:36	9:28:31	13:19:55
20230812_120616	08/12	12:06:52	12:40:43	0:33:51
20230812_124122	08/12	12:41:57	20:37:05	7:55:08
20230812_210904	08/12	21:09:35	6:56:56	9:47:21
20230813_070127	08/13	7:01:58	9:04:33	2:02:35
20230813_090649	08/13	9:07:19	11:00:42	1:53:23
20230813_130704	08/13	13:07:34	15:28:02	2:20:28
20230813_152854	08/13	15:29:26	18:06:09	2:36:43
20230813_193846	08/13	19:39:35	20:26:16	0:46:41
20230813_203334	08/13	20:34:05	4:31:39	7:57:34
20230814_070247	08/14	7:03:16	7:38:14	0:34:58
20230814_074356	08/14	7:44:25	10:04:23	2:19:58
20230814_154152	08/14	15:42:27	17:38:24	1:55:57

20230814_194721	08/14	19:47:50	6:18:46	10:30:56
20230815_064644	08/15	6:47:02	18:46:18	11:59:16
20230816_082802	08/16	8:28:22	12:01:13	3:32:51
20230816_123716	08/16	12:37:55	15:20:45	2:42:50
20230816_153418	08/16	15:34:34	16:17:03	0:42:29
20230816_162627	08/16	16:26:58	17:42:44	1:15:46
20230816_180352	08/16	18:04:21	20:30:45	2:26:24
20230816_210331	08/16	21:04:08	6:43:57	9:39:49
20230817_065318	08/17	6:53:47	7:15:13	0:21:26
20230817_093254	08/17	9:33:23	9:59:15	0:25:52
20230817_100407	08/17	10:04:36	10:29:45	0:25:09
20230817_103124	08/17	10:31:53	10:56:29	0:24:36
20230817_112655	08/17	11:27:34	17:06:34	5:39:00
20230817_171840	08/17	17:19:09	20:11:29	2:52:20
20230817_205825	08/17	20:59:15	8:04:10	11:04:55
20230818_082204	08/18	8:22:20	12:31:15	4:08:55
20230818_132716	08/18	13:27:34	18:12:05	4:44:31
20230818_182654	08/18	18:27:25	20:35:49	2:08:24
20230819_020022	08/19	2:01:19	11:29:07	9:27:48
20230819_124508	08/19	12:46:05	15:12:16	2:26:11
20230819_153749	08/19	15:38:45	17:05:06	1:26:21
20230819_174932	08/19	17:50:28	20:20:14	2:29:46
20230819_231747	08/19	23:18:27	7:01:02	7:42:35
20230820_083812	08/20	8:38:42	10:05:41	1:26:59
20230820_102239	08/20	10:23:24	12:24:20	2:00:56
20230820_130824	08/20	13:09:11	15:56:41	2:47:30
20230820_171340	08/20	17:14:17	17:47:57	0:33:40
20230820_183221	08/20	18:33:26	21:30:56	2:57:30
20230820_213959	08/20	21:40:18	3:56:11	6:15:53
20230821_062516	08/21	6:27:11	10:40:47	4:13:36
20230821_104849	08/21	10:50:55	15:12:38	4:21:43
20230821_152810	08/21	15:30:40	19:20:48	3:50:08

20230821_205512	08/21	20:55:49	13:05:51	16:10:02
20230822_133737	08/22	13:39:03	17:11:22	3:32:19
20230822_182357	08/22	18:24:38	6:08:57	11:44:19
20230823_144153	08/23	14:42:36	21:05:18	6:22:42
20230823_213627	08/23	21:36:48	12:08:39	14:31:51
20230824_120706	08/24	12:07:57	16:44:53	4:36:56
20230824_164806	08/24	16:48:50	21:05:52	4:17:02
20230824_211913	08/24	21:20:27	3:33:00	6:12:33
20230825_225522	08/25	22:56:10	7:02:25	8:06:15
20230826_073826	08/26	7:39:14	8:51:42	1:12:28
20230826_090357	08/26	9:04:17	10:22:58	1:18:41
20230826_102647	08/26	10:27:10	4:09:23	17:42:13
20230827_053357	08/27	5:34:43	17:15:01	11:40:18
20230827_224813	08/27	22:49:08	3:38:20	4:49:12
20230828_033835	08/28	3:39:30	11:05:25	7:25:55
20230828_124303	08/28	12:43:49	17:54:31	5:10:42
20230828_195406	08/28	19:54:55	3:44:33	7:49:38
20230829_071557	08/29	7:17:00	5:01:08	21:44:08
20230830_151212	08/30	15:13:07	21:15:58	6:02:51
20230830_211613	08/30	21:17:16	4:06:43	6:49:27
20230831_200804	08/31	20:08:35	21:23:42	1:15:07
20230831_213235	08/31	21:33:02	22:00:57	0:27:55
20230831_220805	08/31	22:08:53	4:58:44	6:49:51
20230901_061729	09/01	6:17:56	7:12:01	0:54:05
20230901_131623	09/01	13:16:46	4:59:06	15:42:20
20230911_144833	09/11	14:49:31	16:03:47	1:14:16
DL_20230827_182357	08/27	18:24:53	18:47:41	0:22:48
DL_20230827_185154	08/27	18:52:42	22:33:14	3:40:32
DL_20230830_072554	08/30	7:26:48	15:12:00	7:45:12
DL_20230831_070701	08/31	7:07:50	15:40:43	8:32:53
Total				574 hours

7 Conclusion and Discussion

7.1 Conclusion

In this project, I try to discover that if depth information can help to improve the performance of the semantic segmentation model without expanding the parameters too much and avoid the disadvantage of big model that they may have worse performance than the small models with small dataset and short training epochs.

To test my suspicions, I build a model called SDCombo. I make some experiments on this model and compare the performance of SDCombo and control groups. Result of these experiments shows that employing the depth information in a proper way can help to improve the performance of a semantic segmentation model.

Besides the comparing with the original model, more experiments show that this model keeps the advantage of a small model that a small dataset and short training epochs are enough for the training.

7.2 Discussion

Limited to the time and training hardware, there are still some questions I remain to solve.

As shown in the result on classes, performance on some classes is much worse than other classes. I employ the Focal Loss as the loss function, but only get a little improvement on this situation. The original DeepLab shows similar performance on the same classes, so I think maybe employing a backbone to extract the feature more efficiently may help to solve the problem.

The proper format of depth information remains to be an important problem. In this project, HHA shows better performance than raw depth images, but it also has some limitations. Overall, HHA is more suitable for showing the relation between different objects while the raw depth image contains more about the global relations. Also, HHAs need to be calculated from the raw depth image. This means that HHA format is not suitable for a real-time job. It is also a contradiction between a fast-deploy model with an extra data transform.

References

- [1] T. Baltrušaitis, C. Ahuja, and L.-P. Morency. Multimodal machine learning: A survey and taxonomy. *IEEE transactions on pattern analysis and machine intelligence*, 41(2):423–443, 2018.
- [2] H. Cao, C. Tan, Z. Gao, Y. Xu, G. Chen, P.-A. Heng, and S. Z. Li. A survey on generative diffusion model. *arXiv preprint arXiv:2209.02646*, 2022.
- [3] L.-C. Chen, G. Papandreou, I. Kokkinos, K. Murphy, and A. L. Yuille. Semantic image segmentation with deep convolutional nets and fully connected crfs. *arXiv preprint arXiv:1412.7062*, 2014.
- [4] L.-C. Chen, G. Papandreou, I. Kokkinos, K. Murphy, and A. L. Yuille. Deeplab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected crfs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, page 834–848, Mar 2018.
- [5] X. Chen. Depth2HHA-python: Converting depth maps to hha encodings. <https://github.com/charlesCXXK/Depth2HHA-python>, 2018.
- [6] X. Chen, K.-Y. Lin, J. Wang, W. Wu, C. Qian, H. Li, and G. Zeng. *Bi-directional Cross-Modality Feature Propagation with Separation-and-Aggregation Gate for RGB-D Semantic Segmentation*, page 561–577. Dec 2019.
- [7] M. Contributors. MMSegmentation: Openmmlab semantic segmentation toolbox and benchmark. <https://github.com/open-mmlab/msegmentation>, 2020.
- [8] M. Cordts, M. Omran, S. Ramos, T. Rehfeld, M. Enzweiler, R. Benenson, U. Franke, S. Roth, and B. Schiele. The cityscapes dataset for semantic urban scene understanding. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 3213–3223, 2016.
- [9] L. Deng, M. Yang, T. Li, Y. He, and C. Wang. Rfbnet: Deep multimodal networks with residual fusion blocks for rgb-d semantic segmentation. *Cornell University - arXiv, Cornell University - arXiv*, Jun 2019.
- [10] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.

- [11] A. Garcia-Garcia, S. Orts-Escolano, S. Oprea, V. Villena-Martinez, and J. Garcia-Rodriguez. A review on deep learning techniques applied to semantic segmentation. *arXiv preprint arXiv:1704.06857*, 2017.
- [12] S. Gupta, R. Girshick, P. Arbeláez, and J. Malik. Learning rich features from rgb-d images for object detection and segmentation. In *Computer Vision–ECCV 2014: 13th European Conference, Zurich, Switzerland, September 6-12, 2014, Proceedings, Part VII 13*, pages 345–360. Springer, 2014.
- [13] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [14] M. S. Hossain, J. M. Betts, and A. P. Paplinski. Dual focal loss to address class imbalance in semantic segmentation. *Neurocomputing*, 462:69–87, 2021.
- [15] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, et al. Searching for mobilenetv3. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 1314–1324, 2019.
- [16] A. Howard, A. Zhmoginov, L.-C. Chen, M. Sandler, and M. Zhu. Inverted residuals and linear bottlenecks: Mobile networks for classification, detection and segmentation. Jan 2018.
- [17] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*, 2017.
- [18] X. Hu, K. Yang, L. Fei, and K. Wang. Acnet: Attention based network to exploit complementary features for rgb-d semantic segmentation. *Cornell University - arXiv, Cornell University - arXiv*, May 2019.
- [19] J. M. Johnson and T. M. Khoshgoftaar. Survey on deep learning with class imbalance. *Journal of Big Data*, 6(1):1–54, 2019.
- [20] J. Lagos and E. Rahtu. Semsegdepth: A combined model for semantic segmentation and depth completion. In *Proceedings of the 17th International Joint Conference on Computer Vision, Imaging and Computer Graphics Theory and Applications*, Dec 2021.

- [21] Z. Li, K. Kamnitsas, and B. Glocker. Analyzing overfitting under class imbalance in neural networks for image segmentation. *IEEE transactions on medical imaging*, 40(3):1065–1077, 2020.
- [22] R. W. Lienhart. Comparison of automatic shot boundary detection algorithms. In *Storage and retrieval for image and video databases VII*, volume 3656, pages 290–301. SPIE, 1998.
- [23] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár. Focal loss for dense object detection. In *Proceedings of the IEEE international conference on computer vision*, pages 2980–2988, 2017.
- [24] W. Liu, A. Rabinovich, and A. Berg. Parsenet: Looking wider to see better. *arXiv: Computer Vision and Pattern Recognition*, *arXiv: Computer Vision and Pattern Recognition*, Jun 2015.
- [25] J. Long, E. Shelhamer, and T. Darrell. Fully convolutional networks for semantic segmentation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 3431–3440, 2015.
- [26] R. Mottaghi, X. Chen, X. Liu, N.-G. Cho, S.-W. Lee, S. Fidler, R. Urtasun, and A. Yuille. The role of context for object detection and semantic segmentation in the wild. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2014.
- [27] A. Mumuni and F. Mumuni. Data augmentation: A comprehensive survey of modern approaches. *Array*, page 100258, 2022.
- [28] J. Ngiam, A. Khosla, M. Kim, J. Nam, H. Lee, and A. Y. Ng. Multimodal deep learning. In *Proceedings of the 28th international conference on machine learning (ICML-11)*, pages 689–696, 2011.
- [29] T. Pohlen, A. Hermans, M. Mathias, and B. Leibe. Full-resolution residual networks for semantic segmentation in street scenes. *arXiv: Computer Vision and Pattern Recognition*, *arXiv: Computer Vision and Pattern Recognition*, Nov 2016.
- [30] A. Raj, D. Maturana, and S. Scherer. Multi-scale convolutional architecture for semantic segmentation. *Robotics Institute, Carnegie Mellon University, Tech. Rep. CMU-RITR-15-21*, 2015.
- [31] O. Ronneberger, P. Fischer, and T. Brox. U-net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention–*

- MICCAI 2015: 18th International Conference, Munich, Germany, October 5-9, 2015, Proceedings, Part III* 18, pages 234–241. Springer, 2015.
- [32] A. Roy and S. Todorovic. *A Multi-scale CNN for Affordance Segmentation in RGB Images*, page 186–201. Dec 2015.
- [33] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, et al. Imagenet large scale visual recognition challenge. *International journal of computer vision*, 115:211–252, 2015.
- [34] C. Sutton, A. McCallum, et al. An introduction to conditional random fields. *Foundations and Trends® in Machine Learning*, 4(4):267–373, 2012.
- [35] F. Visin, K. Kastner, K. Cho, M. Matteucci, A. Courville, and Y. Bengio. Renet: A recurrent neural network based alternative to convolutional networks. *arXiv: Computer Vision and Pattern Recognition, arXiv: Computer Vision and Pattern Recognition*, May 2015.
- [36] C. Wang, C. Wang, W. Li, and H. Wang. A brief survey on rgb-d semantic segmentation using deep learning. *Displays*, 70:102080, 2021.
- [37] C. Wang, C. Wang, W. Li, and H. Wang. A brief survey on rgb-d semantic segmentation using deep learning. *Displays*, page 102080, Nov 2021.
- [38] W. Wang, J. Dai, Z. Chen, Z. Huang, Z. Li, X. Zhu, X. Hu, T. Lu, L. Lu, H. Li, et al. Internimage: Exploring large-scale vision foundation models with deformable convolutions. *arXiv preprint arXiv:2211.05778*, 2022.
- [39] Z. Wang, E. Wang, and Y. Zhu. Image segmentation evaluation: a survey of methods. *Artificial Intelligence Review*, 53:5637–5674, 2020.
- [40] T. Xiao, Y. Liu, B. Zhou, Y. Jiang, and J. Sun. *Unified Perceptual Parsing for Scene Understanding*, page 432–448. Dec 2017.
- [41] F. Yu and V. Koltun. Multi-scale context aggregation by dilated convolutions. *arXiv: Computer Vision and Pattern Recognition, arXiv: Computer Vision and Pattern Recognition*, Nov 2015.
- [42] Y. Yu, C. Wang, Q. Fu, R. Kou, F. Huang, B. Yang, T. Yang, and M. Gao. Techniques and challenges of image segmentation: A review. *Electronics*, 12(5):1199, 2023.

- [43] B. P. Yuhas, M. H. Goldstein, and T. J. Sejnowski. Integration of acoustic and visual speech signals using neural networks. *IEEE Communications Magazine*, 27(11):65–71, 1989.
- [44] S. Zagoruyko and N. Komodakis. Learning to compare image patches via convolutional neural networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4353–4361, 2015.
- [45] J. Zhang, H. Liu, K. Yang, X. Hu, R. Liu, and R. Stiefelhagen. Cmx: Cross-modal fusion for rgb-x semantic segmentation with transformers. *arXiv preprint arXiv:2203.04838*, 2022.
- [46] H. Zhou, L. Qi, Z. Wan, H. Huang, and X. Yang. Rgb-d co-attention network for semantic segmentation. In *Proceedings of the Asian conference on computer vision*, 2020.

A Code, Deployment and Weight Files

The address to your git project repository: <https://git.cs.bham.ac.uk/projects-2022-23/hxl172.git>

Because of the limitation of the upload files size, I upload the weight files of models in Onedrive.

Model	Weight
SDCombo with HHA	model_20230830_151212.8
SDCombo with Depth	model_20230830_211613.9
DeepLabV3	model_DL_20230831_070701.18
SDCombo with HHA 512	model_20230901_131623.9
SDCombo with HHA 40 epochs	model_20230831_220805.34

Besides the typical models, you can find most raw model weight files in the raw models

The environment which is necessary for the models is written in ReadMe.md. The deployment of DCNv3 is also included in ReadMe.md, which will be needed if you want to use InternImage rewritten by me, or you can use the original InternImage.

The “train_XXX”, “evaluation_XXX” and “demo_XXX” are training, evaluating and demonstrating scripts for corresponding models. “HHA” stands for “SDCombo with HHA”, “Depth” stands for “SDCombo with Depth” and “DL” stands for “DeepLabV3”.

In folder “Utils”, there are some useful tools. In “DataPreparation”, there are scripts for transform depth images to HHA, and tools for pattele. In “Joint”, you can find tools for draw a structure of a model. In ”Segmentation”, you can find tools for “.pth” files.

Other models like SDHead and SDBottleneck are stored in folder “Backup”.